

Rapport de Stage — VulnWatch

Portail intelligent de veille des vulnérabilités des solutions de cybersécurité

Table of Contents

1. Introduction	1
2. Analyse des besoins	1
3. Architecture	2
4. Développement	3
5. Difficultés rencontrées et solutions apportées	4
6. Résultats	5
7. Conclusion.....	5

1. Introduction

Ce rapport présente le travail réalisé dans le cadre d'un stage de développement portant sur la conception et la réalisation de **VulnWatch**, un portail web destiné à automatiser la veille des vulnérabilités affectant les principales solutions de sécurité du marché (pare-feux, antivirus, EDR, etc.).

Le stage s'est déroulé en binôme, avec une répartition initiale des rôles entre backend (collecte, base de données, logique métier) et frontend (interface, dashboard, mise en forme), conformément au principe de découplage recommandé par le guide de réalisation fourni en début de projet. Ce document retrace l'ensemble de la démarche : de l'analyse des besoins jusqu'aux résultats obtenus, en passant par les choix d'architecture, les difficultés rencontrées et les solutions apportées.

2. Analyse des besoins

2.1 Contexte métier

Les équipes de cybersécurité doivent surveiller en permanence l'apparition de nouvelles vulnérabilités (CVE) sur les solutions qu'elles exploitent. Cette information est aujourd'hui dispersée entre plusieurs sources officielles :

- la base **NVD (National Vulnerability Database)**, gérée par le NIST,
- le catalogue **CISA KEV**, qui recense les vulnérabilités activement exploitées,
- les bulletins **PSIRT** publiés directement par chaque éditeur (Fortinet, Palo Alto, Cisco...).

Consulter ces sources une à une, manuellement, chaque jour, est chronophage et rend la priorisation difficile : une CVE avec un score CVSS moyen mais déjà exploitée dans la nature (KEV) est souvent plus urgente à traiter qu'une CVE au score élevé mais purement théorique.

2.2 Besoins fonctionnels retenus

À partir de la fiche de stage, les besoins suivants ont été formalisés :

1. Collecter automatiquement les vulnérabilités depuis plusieurs sources publiques
2. Centraliser ces données dans une base structurée, sans doublons
3. Présenter un tableau de bord synthétique (répartition par éditeur, par sévérité, tendance dans le temps)
4. Permettre une recherche et un filtrage multicritère (éditeur, produit, score CVSS, statut d'exploitation)
5. Fournir une fiche de détail complète par CVE
6. Générer des alertes automatiques sur les nouvelles vulnérabilités critiques
7. Permettre l'export des données (CSV/PDF) pour du reporting
8. Automatiser la synchronisation quotidienne, avec journalisation des exécutions

2.3 Contrat de données (Phase 0)

Conformément au guide de réalisation, un schéma de données commun a été défini avant toute séparation du travail entre backend et frontend, afin que chacun puisse avancer en parallèle : le frontend a d'abord travaillé sur des données fictives respectant ce contrat, en attendant que la collecte réelle soit opérationnelle.

3. Architecture

Le projet suit une architecture en couches, choisie pour sa simplicité pédagogique et sa séparation claire des responsabilités :

Collecteurs (Python) → Normalisation/enrichissement → Base de données (SQLite) → Portail web (FastAPI)

- **Couche collecte** : modules Python indépendants, un par source (NVD, CISA KEV, flux RSS PSIRT), chacun responsable uniquement de récupérer la donnée brute.
- **Couche traitement** : transformation des formats hétérogènes de chaque source vers un schéma unique, calcul de la sévérité à partir du score CVSS, enrichissement avec le statut d'exploitation active (KEV).
- **Couche persistance** : base SQLite via l'ORM SQLAlchemy, choisie pour sa légèreté (aucun serveur à installer) tout en gardant une voie de migration simple vers PostgreSQL si le volume de données venait à grossir.
- **Couche présentation** : API REST FastAPI exposant les données filtrées, consommée par un frontend HTML/CSS/JavaScript utilisant Chart.js pour les visualisations.

Cette séparation stricte a permis au binôme de travailler en parallèle sans se bloquer mutuellement : le frontend construisait ses pages et ses graphiques avec des données

d'exemple pendant que la collecte réelle et la base de données étaient développées côté backend, les deux se rejoignant ensuite pour brancher l'interface sur les vraies données.

4. Développement

4.1 Couche collecte

Trois sources ont été intégrées :

- **NVD** : interrogation de l'API REST officielle par mot-clé vendeur, avec extraction du score CVSS (v3.1 en priorité, retombant sur v3.0 puis v2), de la description, des références et des produits affectés (via les identifiants CPE fournis dans les champs de configuration).
- **CISA KEV** : téléchargement du catalogue complet (fichier JSON unique), puis croisement avec les CVE déjà collectées pour marquer celles activement exploitées.
- **Flux PSIRT** : parsing de deux flux RSS vendeurs (Fortinet et Palo Alto), extraction des identifiants CVE cités dans les bulletins, puis récupération de leurs détails exacts via l'API NVD.

4.2 Couche traitement

Chaque source alimente une fonction de normalisation commune, qui produit un objet respectant strictement le schéma défini en Phase 0 (identifiant, éditeur, produit(s), description, score, sévérité, date, statut de correctif, statut KEV, versions, références, source). Le score et la sévérité sont dérivés directement des métriques CVSS renvoyées par NVD ; lorsque celles-ci sont absentes (CVE très récente, pas encore analysée par NVD), la CVE est explicitement marquée comme "non évaluée" plutôt que de lui attribuer un score de zéro par défaut, afin de ne pas fausser la lecture de sa criticité réelle.

4.3 Couche persistance

Trois tables composent la base : - **vulnerabilities** : une ligne par CVE suivie, avec upsert par identifiant CVE pour garantir l'absence de doublons lors des collectes successives ; - **alerts** : une ligne par alerte générée, avec un mécanisme anti-doublon (une même CVE ne génère jamais deux fois la même alerte) ; - **sync_logs** : historique de chaque exécution de la collecte (déclenchement manuel, automatique ou via l'API), avec statut, compteurs et message d'erreur éventuel.

4.4 Couche présentation

Le backend expose une API REST complète (liste filtrable, détail, statistiques, listes d'éditeurs/produits, export CSV et PDF, gestion des alertes, historique de synchronisation). Le frontend consomme cette API pour afficher un tableau de bord (indicateurs chiffrés, graphique d'évolution sur 14 jours, répartition par sévérité et par éditeur), une liste de vulnérabilités filtrable, une fiche de détail par CVE, et un système de notification des alertes via une cloche dans la barre de navigation.

4.5 Automatisation

La synchronisation quotidienne est programmée via APScheduler, directement intégrée au processus de l'API, qui relance automatiquement la collecte complète chaque nuit à 2h00, sans intervention manuelle. Un déclenchement manuel reste possible via une route dédiée, pour les besoins de test ou de démonstration.

5. Difficultés rencontrées et solutions apportées

5.1 Sévérité toujours affichée comme "Faible"

Difficulté : après le branchement sur les vraies données, toutes les CVE s'affichaient avec un score de 0.0 et une sévérité "Faible", quelle que soit leur gravité réelle.

Diagnostic : la fonction de conversion côté frontend recalculait systématiquement la sévérité à partir du score, sans jamais tenir compte de la valeur déjà déterminée côté backend. Un score à zéro (dû à des CVE pas encore analysées par NVD) entraînait donc un classement automatique et erroné en "Faible" pour l'ensemble des résultats.

Solution : la fonction de conversion a été corrigée pour faire confiance à la sévérité calculée côté backend, et une catégorie distincte "Non évaluée" a été introduite pour les CVE sans score CVSS, avec un affichage explicite ("N/A") plutôt qu'un score fictif de zéro.

5.2 Limite de plage de dates de l'API NVD

Difficulté : la collecte échouait silencieusement pour certains éditeurs sans message d'erreur clair.

Diagnostic : l'API NVD impose une limite stricte de 120 jours d'écart maximum entre les paramètres de date de début et de fin de recherche ; la configuration initiale demandait 180 jours.

Solution : ajustement du paramètre à 120 jours, conforme à la documentation officielle de l'API.

5.3 Base de données non migrée après modification du schéma

Difficulté : l'ajout de nouvelles colonnes (par exemple le champ `unscored`, ou la table `sync_logs`) provoquait des erreurs `no such column` lors de l'exécution.

Diagnostic : SQLAlchemy ne modifie jamais une base de données existante ; il ne fait que créer les tables absentes.

Solution : documentation claire de la procédure à suivre en développement (suppression du fichier de base avant relance) ; une vraie migration de schéma (via un outil comme Alembic) serait la solution recommandée en environnement de production.

5.4 Mise en cache du navigateur

Difficulté : des corrections apportées aux fichiers JavaScript ne se répercutaient pas visuellement sur le site, laissant penser à tort que le code restait inchangé.

Solution : identification du cache navigateur comme cause probable, résolue par un rechargement forcé et par l'ajout de paramètres de version sur les balises de script.

5.5 Répartition des rôles en pratique

Difficulté : la séparation stricte backend/frontend prévue initialement a dû être assouplie au cours du projet, l'un des deux stagiaires ayant dû intervenir sur les deux couches pour maintenir l'avancement global.

Solution : maintien rigoureux du contrat de données défini en Phase 0 comme référence commune, permettant de continuer à faire évoluer les deux couches de façon cohérente malgré ce changement d'organisation.

6. Résultats

Le portail VulnWatch est aujourd'hui fonctionnel et démontre :

- une collecte réelle et automatisée depuis trois sources externes (NVD, CISA KEV, deux flux PSIRT vendeurs),
- une base de données structurée sans doublons, alimentée de façon incrémentale,
- un tableau de bord interactif avec indicateurs chiffrés et graphiques de tendance,
- une liste de vulnérabilités filtrable selon cinq critères combinables,
- une fiche de détail complète par CVE, incluant l'ensemble des produits affectés,
- un système d'alertes fonctionnel, avec anti-doublon, visible directement dans l'interface,
- un export CSV et PDF respectant les filtres actifs,
- une synchronisation quotidienne automatique, journalisée et consultable via l'API.

7. Conclusion

Ce projet a permis de mettre en pratique l'ensemble du cycle de vie d'une application de veille sécurité : manipulation d'API REST et de flux publics réels, conception d'une architecture en couches, persistance de données via un ORM, exposition d'une API filtrable, et construction d'une interface web complète. Au-delà des compétences techniques, le travail en binôme sur un contrat de données partagé a été une mise en application concrète des bonnes pratiques de collaboration en développement logiciel, y compris lorsque la répartition initiale des rôles a dû être ajustée en cours de route.

Les principales pistes d'amélioration identifiées pour la suite du projet concernent l'ajout de sources de données supplémentaires (Cisco PSIRT, CVE.org), la mise en place d'une véritable authentification utilisateur, et l'introduction d'un outil de migration de schéma (Alembic) pour fiabiliser les évolutions futures de la base de données en environnement de production.